Disease Prediction System in Python using ML

Project by:
PARTH KANDORIA 221030162
ABHINAV PATHANIA 221030171

Submitted to:
Dr. Nishant Jain

Submission Date:
19/11/2023

Purpose of the Project:

The **purpose** of a symptom-based disease diagnosis project using machine learning is to provide a tool that can assist individuals in gaining insights into potential health issues based on their reported symptoms. The main goals and purposes of this project are as follows:

1. **Early Detection and Awareness**:

By inputting their symptoms into the system, users can get early insights into potential diseases or health conditions. This can increase awareness about their health and encourage them to seek medical advice or take appropriate action.

2. **Accessibility**:

The project provides an easily accessible and user-friendly platform for users to input their symptoms and receive preliminary information about possible diseases. This can be especially useful in scenarios where individuals might not have immediate access to healthcare professionals.

3. **Information and Education**:

The project can provide users with information about various diseases, their common symptoms, and potential treatments. This can contribute to health education and help users make more informed decisions about their well-being.

4. **Emergency Situations**:

In some cases, the system can help identify critical or emergency situations where immediate medical attention is required. This can be a life-saving aspect of the project.

5. **Feedback Mechanism**:

Users can provide feedback on the system's accuracy, which can be used to continually improve the model and its predictions. This feedback loop can enhance the system's performance over time.

6. **Resource Optimization**:

By offering a preliminary assessment of symptoms, the project can help users determine if they need to consult a healthcare professional. This can potentially reduce unnecessary doctor visits while ensuring that those with concerning symptoms seek medical care promptly.

7. **Research and Data Analysis**:

The project can generate valuable data that can be used for research and data analysis in the field of healthcare. This data can potentially contribute to a better understanding of the prevalence of certain diseases and the symptoms associated with them.

It's important to note that while this project can be a valuable tool for initial health assessments, **it should never replace professional medical advice, diagnosis, and treatment.** The primary purpose is to assist users in making more informed decisions about their health and to encourage them to seek appropriate medical care when necessary.

Methods/Solutions

Introduction:

Healthcare is a vital sector where technology plays a crucial role in improving diagnostic and predictive capabilities. This project focuses on disease prediction based on symptoms using machine learning techniques. Several methods and solutions are available globally to address this challenge.

1. Dataset Collection:

a. Public Datasets:

- Utilizing publicly available medical datasets from reputable sources (e.g., UCI Machine Learning Repository, Kaggle).
- Ensuring the datasets cover a diverse range of diseases and symptoms for robust model training.

b. Custom Datasets:

- Collaborating with healthcare providers to gather specific datasets relevant to the target population or region.
- Ensuring ethical considerations and patient privacy are maintained during data collection.

2. Feature Engineering

a. Symptom Encoding:

- Using techniques like one-hot encoding or label encoding to convert categorical symptom data into a format suitable for machine learning algorithms.
- Handling unknown or rare symptoms appropriately in the encoding process.

b. Data Augmentation:

- Employing data augmentation techniques to artificially increase the size of the dataset.
- Generating additional synthetic data points to enhance model generalization.

3. Model Development

a. Machine Learning Algorithms:

- Employing classification algorithms such as Random Forest, Support Vector Machines (SVM), or Neural Networks for disease prediction.
- Evaluating and comparing the performance of different algorithms.

b. Ensemble Methods:

- Building ensemble models to combine predictions from multiple base models, enhancing overall prediction accuracy.
- Utilizing techniques like bagging or boosting to create diverse ensembles.

4. Model Evaluation

a. Cross-Validation:

- Implementing cross-validation techniques to assess the model's performance across different subsets of the dataset.
- Ensuring the model's robustness and generalization capability.

b. Metrics Selection:

- Choosing appropriate evaluation metrics (e.g., accuracy, precision, recall) based on the specific requirements of the disease prediction task.
- Balancing sensitivity and specificity for a comprehensive evaluation.

5. Deployment

a. Web-based Application:

- Developing a user-friendly web-based application for easy access to the disease prediction model.
- Integrating real-time symptom input for instant predictions.

b. Mobile Application:

- Creating a mobile application for on-the-go disease prediction.
- Ensuring compatibility with various mobile platforms.

6. Continuous Improvement

a. Feedback Mechanism:

- Implementing a feedback mechanism to collect user feedback on the predictions.
- Iteratively updating the model based on new data and user experiences.

b. Adapting to Emerging Diseases:

- Designing the system to adapt to emerging diseases by updating the dataset and model when new information becomes available.
- Maintaining a proactive approach to public health challenges.

Methodology

1. Dataset Collection:

Collect a comprehensive dataset containing information about diseases and associated symptoms. Ensure diversity in the types of diseases and symptoms represented.

2. Data Preprocessing:

- Load the dataset into a Pandas DataFrame.
- Separate the features (symptoms) and target labels (diseases).
- Encode categorical features (symptoms) using Label Encoding.
- Split the dataset into training and testing sets using `train_test_split`.

3. Model Training:

- Initialize a Random Forest Classifier.
- Train the classifier using the training dataset (`fit(X_train, y_train)`).
- Utilize the RandomForestClassifier's ability to handle both small and large feature sets during training.

4. Feature Encoding for User Input:

- Define a function, `predict_disease`, to handle predictions based on user input.
- Use Label Encoders saved during training to encode user-provided symptoms.
- Check the number of symptoms provided:
- If 5-6 symptoms are given, predict diseases based on those symptoms.
- If more than 6 symptoms are provided, predict diseases using all symptoms.
-

5. User Interaction:

- Prompt the user to input their symptoms.
- Ensure that the user provides at least 5 symptoms; otherwise, raise a ValueError.

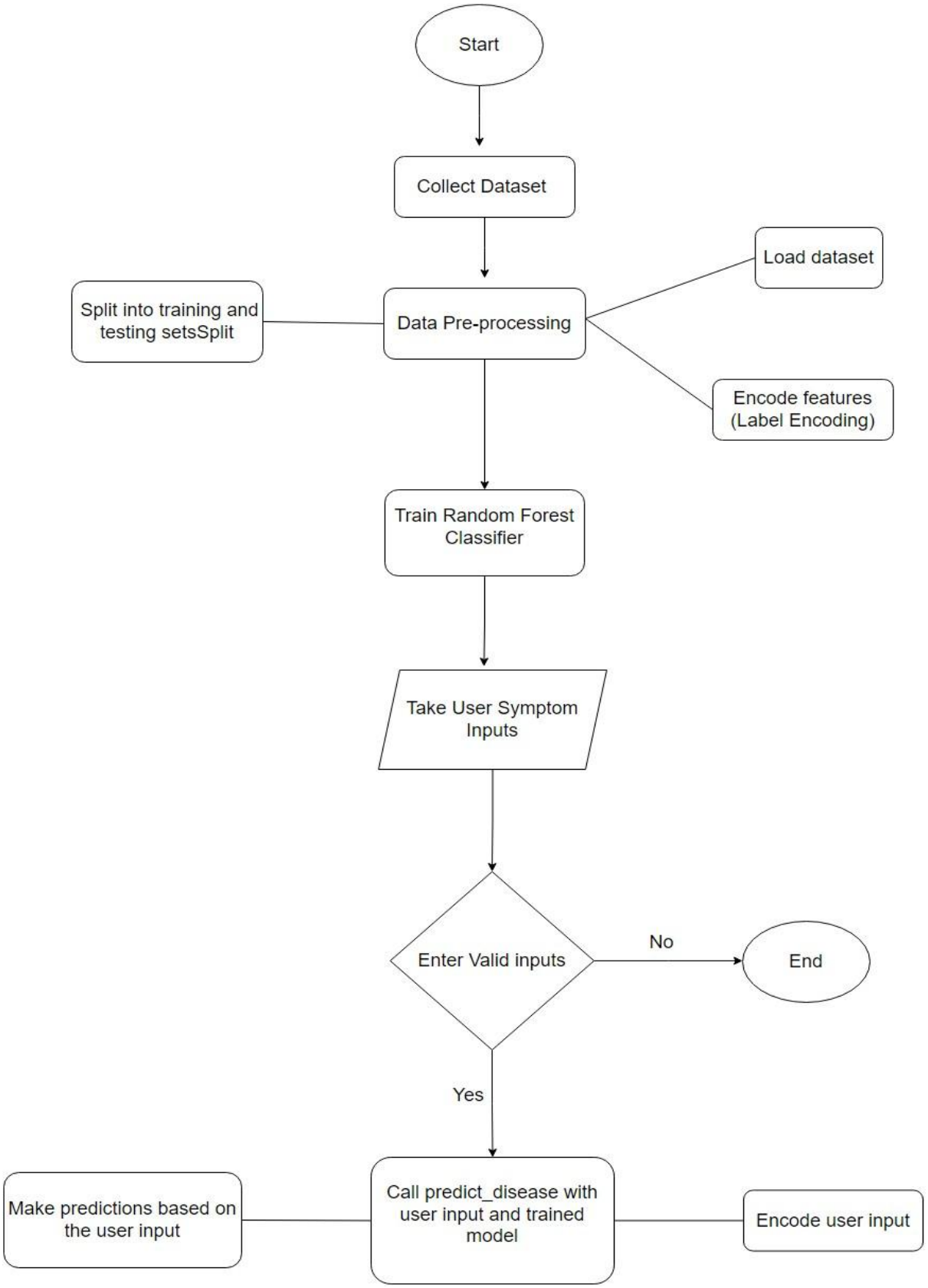
6. Prediction and Output:

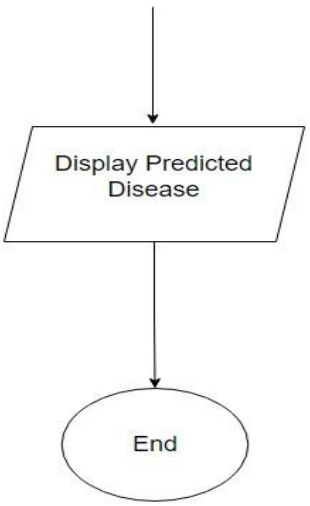
- Call the `predict_disease` function with the user's input, trained classifier, and saved Label Encoders.
- Display the predicted disease based on the provided symptoms.

7. Exception Handling:

- Implement exception handling to capture errors, such as insufficient symptoms or unexpected issues during prediction.

Flowchart


Start
Collect Dataset
Load dataset
Split into training and testing setsSplit
Data Pre-processing
Encode features (Label Encoding)
Train Random Forest Classifier
Take User Symptom Inputs
Enter Valid inputs
No
End
Yes
Make predictions based on the user input
Call predict_disease with user input and trained model
Encode user input



Dataset Details

1. Dataset Source:

The dataset used in this disease prediction project was collected from UCI Machine Learning Repository and Kaggle.

2. Dataset Overview:

A. <u>Size:</u>

- The number of instances (rows) are : 10
- The number of features (columns) in the dataset are : 6

B. <u>Attributes:</u>

- The dataset includes the following features (Symptoms): Symptom1,
- The target variable is the disease label.

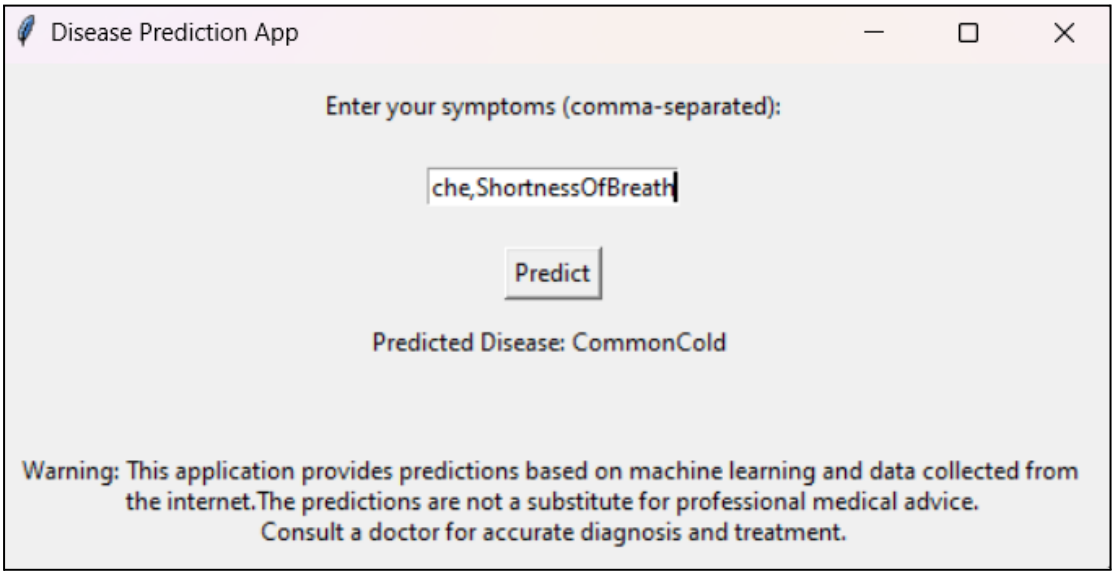
C. <u>Target Variable:</u>

The target variable is "Disease," representing the type of disease associated with the given symptoms.

D. Data Types:

The symptoms are categorical features that have been transformed into numerical format using LabelEncoder.

E. Train-Test Split:

The dataset is split into training and testing sets using train_test_split with a test size of 20%.

Snapshots of GUI